compare_fastq.py 사용법 및 코드 설명서

대상 파일: `lab11/src/compare_fastq.py`

1. 프로그램의 목적

트리밍(fastp) **전** FASTQ 파일과 트리밍 **후** FASTQ 파일을 각각 읽어 다음 다섯 가지 항목을 계산하고, 두 파일의 값을 한 표에 나란히 출력하여 비교한다.

- 전체 read 개수
- 평균 read 길이
- 전체 평균 Phred 품질 점수
- 가장 짧은 read 길이
- 가장 긴 read 길이

전체 평균 품질은 `qual_summary.py`와 같은 방법, 즉 **read마다 평균 품질을 구한 뒤 그 값들을 다시 평균** 내는 방법으로 계산한다.

2. 실행 방법

`lab11` 폴더에서 다음과 같이 실행한다.

```
python3 src/compare_fastq.py data/SRR1553607_1.fastq data/trimmed.fastq
```

형식:

```
python3 src/compare_fastq.py <트리밍 전 FASTQ> <트리밍 후 FASTQ>
```

- 첫 번째 인자(`sys.argv[1]`)는 트리밍 전 파일, 두 번째 인자(`sys.argv[2]`)는 트리밍 후 파일이다. 순서를 바꾸면 "차이" 열의 부호가 반대로 나온다.
- 인자 개수를 검사하는 코드는 없다. 인자를 하나만 주면 `sys.argv[2]`에서 `IndexError`가 난다.
- 두 파일을 모두 메모리에 읽으므로 파일이 크면 시간이 조금 걸린다 (아래 실행에서 약 8초).

3. 입력 파일

FASTQ 형식의 텍스트 파일 두 개이다. read 하나는 **4줄**(이름, 서열, `+` 줄, 품질 문자열)로 이루어진다. `read_fastq()`는 파일이 정확히 4줄 단위라고 가정한다.

파일	내용
data/SRR1553607_1.fastq	트리밍 전 원본 read (forward)
data/trimmed.fastq	fastp로 트리밍한 결과

4. 출력 결과

화면에 탭(`\t`)으로 구분된 표를 출력한다. 파일로 저장하지는 않는다. 숫자는 `round(값, 2)`로 소수점 둘째 자리까지 반올림한다.

`lab11` 폴더에서 실제로 실행한 결과는 다음과 같다.

```
$ python3 src/compare_fastq.py data/SRR1553607_1.fastq data/trimmed.fastq
트리밍 전: data/SRR1553607_1.fastq
트리밍 후: data/trimmed.fastq

항목  트리밍 전   트리밍 후   차이(후-전)
전체 read 개수  203445  188357  -15088
평균 read 길이  101.0   98.94   -2.06
전체 평균 품질    33.45   34.95   1.5
최소 read 길이  101 50  -51
최대 read 길이  101 101 0
```

보기 좋게 정리하면 다음과 같다.

항목	트리밍 전	트리밍 후	차이(후-전)
전체 read 개수	203445	188357	-15088
평균 read 길이	101.0	98.94	-2.06

항목	트리밍 전	트리밍 후	차이(후-전)
전체 평균 품질	33.45	34.95	1.5
최소 read 길이	101	50	-51
최대 read 길이	101	101	0

결과 확인:

- 전체 평균 품질은 `qual_summary.py`의 결과(트리밍 전 33.4497..., 트리밍 후 34.9476...)를 반올림한 값과 같다.
- read 개수(203445, 188357)는 `doc/fastp.json`의 `total_reads` 값과 같다.
- read 개수, 평균/최소/최대 길이는 `awk`로 따로 센 값과도 같았다.

5. 결과 해석

- **read 개수 감소 (-15088, 약 7.4%)**: fastp가 품질이 낮거나, 잘라 낸 뒤 너무 짧아진 read를 버렸기 때문이다.
- **평균 길이 감소, 최소 길이 50**: 트리밍 전에는 모든 read가 101 bp였지만, 트리밍 후에는 read 끝의 품질 낮은 부분이나 어댑터가 잘려 길이가 다양해졌다. 최소 길이가 정확히 50인 것은 fastp를 `-l 50` 옵션(50 bp보다 짧아진 read는 버림)으로 실행했기 때문이다. `doc/fastp.json`의 `command` 항목에 실행 명령이 기록되어 있다: `fastp -i data/SRR1553607_1.fastq -o data/trimmed.fastq -q 20 -l 50 ...` (`-q 20`: 품질 점수 20 이상을 좋은 염기로 본다).
- **최대 길이 101 유지**: 잘라 낼 부분이 없던 read는 원래 길이 그대로 남았다.
- **평균 품질 상승 (+1.5)**: 품질이 낮은 염기와 read가 제거되었기 때문에 남은 데이터의 평균 품질이 올라갔다.

6. 코드 설명

6.1 `read_fastq(filename)` — FASTQ 파일 읽기

```
def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()
    for i in range(0, len(lines), 4):
        name = lines[i].rstrip()[1:]
        seq = lines[i + 1].rstrip()
        qual = lines[i + 3].rstrip()
        records.append((name, seq, qual))
    return records
```

다른 lab11 스크립트(`qual_summary.py` 등)에 있는 함수와 똑같다. 파일의 모든 줄을 읽고, 4줄씩 건너뛰면서 `(이름, 서열, 품질)` 튜플을 만들어 리스트로 돌려준다.

6.2 `mean_quality(qual)` — read 하나의 평균 품질

```
def mean_quality(qual):
    scores = []
    for ch in qual:
        scores.append(ord(ch) - 33)
    return sum(scores) / len(scores)
```

`qual_summary.py`와 같은 함수이다. 품질 문자 하나하나를 `ord(ch) - 33`으로 Phred 점수로 바꾸고 (예: `'B'` → 66 - 33 = 33, `'J'` → 74 - 33 = 41), 점수들의 합을 개수로 나누어 평균을 구한다.

6.3 `summarize(filename)` — 파일 하나의 다섯 가지 항목

```
def summarize(filename):
    reads = read_fastq(filename)

    lengths = []      # read마다의 길이
    qualities = []    # read마다의 평균 품질
    for name, seq, qual in reads:
        lengths.append(len(seq))
        qualities.append(mean_quality(qual))

    count = len(reads)
    mean_length = sum(lengths) / count
    mean_qual = sum(qualities) / count
```

```
    min_length = min(lengths)
    max_length = max(lengths)
    return count, mean_length, mean_qual, min_length, max_length
```

이 프로그램의 핵심 함수이다. 트리밍 전 파일과 후 파일에 똑같은 계산을 해야 하므로, 계산 부분을 함수 하나로 묶고 두 번 부른다.

1. `reads = read_fastq(filename)`: 파일을 읽어 read 리스트를 만든다.
2. 반복문에서 read 하나마다 두 값을 리스트에 모은다.
3. `len(seq)`: 그 read의 길이 → `lengths`
4. `mean_quality(qual)`: 그 read의 평균 품질 → `qualities`
5. 모은 리스트로 다섯 항목을 계산한다.

항목	계산	설명
전체 read 개수	len(reads)	리스트에 든 read 수
평균 read 길이	sum(lengths) / count	길이 합 ÷ read 수
전체 평균 품질	sum(qualities) / count	read별 평균 품질의 합 ÷ read 수
최소 read 길이	min(lengths)	길이 리스트에서 가장 작은 값
최대 read 길이	max(lengths)	길이 리스트에서 가장 큰 값

1. `return count, mean_length, ...`: 다섯 값을 쉼표로 이어 돌려주면 하나의 튜플이 된다. 순서는 위 표의 순서와 같다.

주의: 파일에 read가 하나도 없으면 `count`가 0이 되어 `ZeroDivisionError`가, `min()`/`max()`에서 `ValueError`가 난다. 이를 막는 코드는 없다.

6.4 "read별 평균의 평균"이란

`mean_qual`은 모든 염기의 점수를 한꺼번에 평균 낸 값이 아니라, read마다 평균을 먼저 구한 뒤 그 평균들을 다시 평균 낸 값이다. `qual_summary.py`와 같은 결과를 얻기 위해 이 방법을 썼다.

- 트리밍 전에는 모든 read가 101 bp이므로 두 방법의 값이 같다.
- 트리밍 후에는 read 길이가 서로 달라서 두 방법의 값이 조금 다를 수 있다. 짧은 read도 긴 read와 같은 비중(1표)으로 계산되기 때문이다.

6.5 메인 부분 — 두 파일 계산과 표 출력

```
before_file = sys.argv[1]
after_file = sys.argv[2]

before = summarize(before_file)
after = summarize(after_file)

labels = ["전체 read 개수", "평균 read 길이", "전체 평균 품질", "최소 read 길이", "최대 read 길이"]

print("트리밍 전: " + before_file)
print("트리밍 후: " + after_file)
print()
print("항목\t트리밍 전\t트리밍 후\t차이(후-전)")
for i in range(len(labels)):
    b = before[i]
    a = after[i]
    print(labels[i] + "\t" + str(round(b, 2)) + "\t" + str(round(a, 2)) + "\t" + str(round(a - b, 2)))
```

1. 명령줄 인자 두 개를 받아 각 파일에 `summarize()`를 부른다. `before`와 `after`에는 다섯 값이 든 튜플이 들어간다.
2. `labels` 리스트에는 항목 이름을 `summarize()`가 돌려주는 순서와 **같은 순서**로 적어 둔다.
3. `for i in range(len(labels))`: `i`가 0부터 4까지 바뀌며, `labels[i]`, `before[i]`, `after[i]`가 같은 항목을 가리킨다. 그래서 한 줄에 같은 항목의 트리밍 전 값, 후 값, 차이를 나란히 출력할 수 있다.
4. `round(값, 2)`로 소수점 둘째 자리까지 반올림하고, `str()`로 문자열로 바꾼 뒤 `+`와 `"\t"`(탭)로 이어 붙인다. 정수인 항목(개수, 최소/최대 길이)은 `round()`를 해도 정수 그대로 출력된다.
5. 차이는 `a - b`(후 - 전)이므로, 음수이면 트리밍 후 줄어든 것이고 양수이면 늘어난 것이다.

7. 사용한 기본 기능 정리

기능	이 프로그램에서 하는 일
for 반복문	read 하나씩 처리, 표의 항목 하나씩 출력

기능	이 프로그램에서 하는 일
len()	read 길이, read 개수, 품질 점수 개수
sum()	길이 합, 품질 점수 합, read별 평균 품질 합
min() / max()	가장 짧은 / 가장 긴 read 길이
ord(ch) - 33	Phred+33 문자를 점수로 변환
round() / str()	출력할 숫자 정리, 문자열 이어 붙이기